Supplementary Material for
Multi-Period Modular Facility and Network Expansion with Endogenous Throughput
Data and Computational Reproduction

This supplement describes the data and computational workflow of CCS_FNE_LRH, a Python implementation of multi-period carbon capture, transport and storage network planning. It provides a practical guide to the input files, data preparation, experiment design, direct Gurobi, primal-heuristic and Lagrangian solution procedures, and saved results. The repository combines a national candidate network with unit-level capture options, storage-subsite capacities and costs, and a catalogue of pipeline classes.

The standard reproduction route begins with six processed CSV files in the project root. The loader constructs the network and facility parameters, and the cost module converts them into period-specific coefficients. A feasible-solution heuristic and an original-model linear relaxation initialize the Lagrangian procedure. Proximal Bundle iterations then update certified lower bounds, while neighborhood searches improve audited feasible plans. Single-case runs export settings, bounds, decisions and logs; batch outputs are described below.

I. Computational Environment and Reproduction Workflow

A. Software and Directory Structure

The optimization code uses Python and the Gurobi Python interface, with NumPy, pandas, NetworkX and openpyxl for numerical arrays, data handling, network operations and workbook output. Matplotlib supports optional result visualization; pytest is used for verification. The local solver environment used for the entry-point checks has Python 3.10 and Gurobi 12.0.3. Gurobi requires a valid license. The spatial preprocessing stages have additional dependencies, including scientific and geographic libraries and, for storage-subsite construction, ArcGIS Pro with ArcPy. These spatial tools are not required when solving from the supplied processed CSV files.

The project root contains the input tables, shared model and cost routines, and direct Gurobi and primal heuristic (PH) solvers. The direct Gurobi entries are `run_gurobi.py` for one case and `CCS_FNE.py` for a batch; the corresponding PH entries are `run_PH.py` and `CCS_FNE_PH.py`. The `LRforFNE/` directory contains the Lagrangian implementation, entry scripts and tests, with `run_lrh.py` and `CCS_FNE_LR.py` as its editable single-case and batch entries. All six scripts can be run directly in an IDE or by passing their paths to Python. Their default data and output paths are relative to their own files. Command examples below assume the project root as the working directory.

B. Reproduction Sequence

Table S.1 summarizes the computational chain. The preprocessing scripts are needed when changing the underlying spatial or engineering inputs. Routine optimization starts from the processed root-level tables and does not regenerate them.

Table S.1—Main reproduction sequence

Stage	Entry point or module	Output
Data preparation	`source_cluster/`, `storage_cost/`, `triangle network/`	Capture hubs, storage subsites, candidate network and costs.
Instance construction	`read.py`, `experiment_setup.py`, `cost_amortize.py`	Network parameters, capture targets and period-specific costs.
Single PH case	`run_PH.py`	A variable workbook, solve logs and construction maps.
Batch PH cases	`CCS_FNE_PH.py`	A batch CSV summary with bounds, status and timing.
Single Lagrangian case	`LRforFNE/run_lrh.py`	Bundle JSON results and a variable workbook.
Batch Lagrangian cases	`LRforFNE/CCS_FNE_LR.py`	A batch CSV summary and per-case JSON results.
Single Gurobi case	`run_gurobi.py`	A variable workbook, solver log and construction maps.
Batch Gurobi cases	`CCS_FNE.py`	A CSV summary of final bounds, gap, status and bound snapshots.
Visualization	`plot_results.py`	Construction maps or bound plots from saved decisions and logs.

Note: Preprocessing outputs are written to their own directories. Updated tables must be explicitly synchronized with the six model inputs in the project root.

II. Input Data and Parameter Construction

A. Network and Facility Tables

The current network has 2,517 nodes: 580 source hubs, 271 storage subsites and 1,666 transshipment nodes. Its 14,372 directed arcs represent 7,186 bidirectional candidate corridors. Capture options comprise 3,069 hub–unit combinations. Transportation costs cover all 12 pipeline classes on every directed arc, giving 172,464 arc–class records. Table S.2 lists the six files read by the optimization workflow; counts exclude header rows.

The loader checks node and arc identifiers, facility coverage and completeness of the arc–class cost catalogue. Source capacity, costs and activation decisions remain specific to each capture unit, even when several units share a hub. Storage parameters refer directly to the physical subsite connected to the transportation network. The current subsite data provide a total maximum of 7,573 injection wells, with site-specific limits between 5 and 49.

B. Spatial and Engineering Preparation

The source preprocessing starts from `source_cluster/source_unit_location_cost.csv`, containing 3,069 units at 1,115 source locations. The formal clustering script, `cluster.py`, applies DBSCAN with a 20 km Haversine adjacency threshold and a minimum cluster size of one. The threshold determines neighborhood connectivity rather than a maximum cluster diameter. Units are reassigned to 580 hubs, whose coordinates are the arithmetic means of their constituent unit-row coordinates. Capture capacities and cost attributes are

Table S.2—Processed optimization inputs

File	Records	Main fields and interpretation
`nodes.csv`	2,517	`I`, `Longitude`, `Latitude`; WGS84 node coordinates. Prefixes H, P and I identify source hubs, storage subsites and transshipment nodes.
`arcs.csv`	14,372	`i`, `j`, `distance`; ordered endpoints and candidate-link length in km.
`hub_unit_location_cost.csv`	3,069	Hub and unit identifiers, coordinates, annual capture capacity, total investment and operating cost per tonne.
`sink_subsite_cost.csv`	271	Subsite identifier, maximum wells, annual capacity per well, site and well costs in USD2020, operating schedules and lifecycle cash flows.
`pipeline_capacity.csv`	12	`d`, `cap`; pipeline-class identifier and annual transport capacity.
`pipeline_cost.csv`	172,464	Directed-arc endpoints, pipeline-class identifier, total investment in USD2020 and operating cost in USD2020 per tonne.

Note: The historical column name `CO2 flow rate(kg/s)` in the pipeline cost file contains class identifiers such as `p4`; the loader treats it as an index, not a numerical flow in kg/s.

retained. The supplied unit-level input is the starting point for this workflow; its upstream collection and cost-estimation procedures are outside the supplied source-preprocessing scripts.

Storage preprocessing partitions the usable basin geometry into 271 subsites in 14 basins with matched boundaries. The subsite routine uses a local equal-area projection, weighted Voronoi partitioning and compactness adjustments, with an 8,000 km^2 maximum area and a 1.03 within-basin maximum-to-minimum area ratio. Representative points lie within the resulting polygons. Basin storage resources and annual injection allowances are allocated using the recorded area rules. Annual well capacity is an engineering proxy, computed as 365 times the smaller of the formation-based daily rate and 3,660 tonnes per day. The maximum well count is the integer part of the subsite annual allowance divided by this annual well capacity. The optimization model uses these well-count limits and annual per-well capacities; allocated geological resources are not imposed as a separate cumulative-storage constraint.

The candidate-network generator reads the hub and subsite locations, adds auxiliary transshipment nodes and constructs a mainland-constrained Delaunay network. Boundary filtering and connectivity repair precede export of paired directed arcs. Its adaptive placement uses a 55 km coverage radius, followed by pruning of terminal auxiliary branches. Arc lengths represent projected candidate connections rather than surveyed pipeline routes.

The pipeline-cost script expands each directed arc over 12 classes. Its reference flow values are stored in the code and documented against the accompanying `CO2_T_COM_ver4_final_20230726.xlsm` workbook; they already incorporate a 90% capacity factor. The costing length is the candidate-link length multiplied by 1.12, with a further 15-mile allowance when the destination is a storage subsite. Directional costs can therefore differ even when the two geometric arc lengths are equal. Annual operating cost is set to 2.5% of investment and divided by the reference annual throughput to obtain a per-tonne coefficient. The exported investment is the total capital amount; the optimization cost module applies its own financing and discounting treatment.

C. Units and Cost Timing

Annual capture, injection and transportation quantities are measured in kt CO_2/year. Source, pipeline and well capacities expressed in tonnes per year are divided by 1,000 on loading. Dollar-denominated investments and fixed cash-flow amounts are likewise divided by 1,000. Variable costs in dollars per tonne retain their numerical values, so multiplying them by flows in kt/year gives costs in thousands of dollars per year. A period's physical quantity is its annual flow multiplied by the period duration. The input well capacity is already annual and receives no additional daily-to-annual conversion in the model loader.

All storage and pipeline costs, including lifecycle cash flows, are in constant 2020 US dollars (USD2020). The pipeline CSV labels `$` and `$/tCO2` denote USD2020 and USD2020 per tonne. Loading scales fixed monetary amounts to kUSD2020 without further price-year conversion.

The planning horizon is 30 years. The default discount rate is 7%, the financing rate used in the capital-recovery factor is 9%, and the default technical lifetime is 30 years for capture facilities, storage sites, wells and pipelines. Investment coefficients include the present value of annual capital payments through the planning horizon plus the discounted positive outstanding capital balance at its end. Operating-flow coefficients sum annual discount factors within the applicable period.

Storage costs are constructed as a NETL engineering-analogue scenario in constant 2020 US dollars, using the local saline-storage workbook, basin information, reference data and electricity-tariff tables. Tariffs are dated monthly snapshots, applied with region, voltage and uniform-load assumptions rather than measured annual-average prices for each site. The processed file uses schema version 3 and 30-year operating-age schedules. Its JSON lifecycle field records pre-operation expenses, closure at the common end of the planning horizon and 50 years of post-injection site care. The cost module incorporates these timed cash flows as well as site and well fixed operating costs.

III. Experiment Design and Solution Procedures

A. Shared Cases and Single-Case Settings

The batch design is defined in `experiment_setup.py`. Table S.3 gives its five dimensions, whose Cartesian product contains 224 model cases. The 4-class catalogue is a subset of the complete 12-class catalogue. The labels `p24a/p24b` and `p30a/p30b` identify distinct capacity classes, so the catalogues should not be interpreted as lists of mutually distinct nominal diameters.

Table S.3—Shared experimental grid

Dimension	Values
Number of periods	5, 10, 15, 30, corresponding to periods of 6, 3, 2 and 1 years.
Terminal annual capture target	500,000 or 2,000,000 kt CO_2/year, equivalent to 0.5 or 2 Gt/year.
Pipeline installation limit NP	1 or 3, counted separately for each directed arc over the full horizon and all selected classes.
Pipeline catalogue	4 classes: p4, p6, p20, p48; or all 12 classes, adding p10, p12, p16, p24a, p24b, p30a, p30b and p36.
Capture-target profile	Linear; exponential with coefficient 1, 2.5 or 4; logarithmic with coefficient 2, 15 or 500.

Note: The targets are already in model flow units. NP is a full-horizon directed-arc limit, not an annual or per-period construction allowance.

By default, the linear, exponential and logarithmic profiles share the same terminal annual target, while their cumulative capture requirements differ. Setting `NORMALIZE_CAPTURE_TARGET=True` rescales each generated nonlinear sequence to the sum of the corresponding linear sequence. Because periods within a case have equal length, this also equalizes cumulative target quantities. Under this option, the configured maximum is the linear reference terminal value; it need not equal the rescaled nonlinear terminal value. A single-case explicit `CAPTURE_AMOUNT_SEQUENCE` takes precedence over generated curves and may be nonmonotone.

The editable Lagrangian single-case entry defaults to five periods, NP equal to three, 12 pipeline classes, a linear profile and a terminal annual target of 1,000,000 kt CO_2/year, independently of the batch grid. The Lagrangian interfaces support only proximal Bundle: `METHOD`, CLI `--method` and Python `method` accept only `bundle`. Both editable Lagrangian entries default to 2,400 seconds for feasible initialization, an unlimited multiplier LP, and 1,200 seconds for the Lagrangian phase. The maximum of 100 oracle evaluations is separate from the runtime limit. These editable settings are not measured runtimes.

B. Lagrangian Initialization and Bound Computation

The original mixed-integer model is defined by `CCS_FNE.build_milp`. Capture-unit and storage-site activation decisions are binary. New wells and new pipelines are nonnegative integer counts; annual capture, injection and transport flows are continuous. Construction remains available in subsequent periods. Facility capacities, site-specific well limits, annual capture requirements, node balances and full-horizon pipeline-count limits link the decisions over time.

The Lagrangian workflow first obtains a feasible plan through the shared primal heuristic. The default P1 backend is RSS; subsequent searches revise facility timing, pipeline construction and flows. This initialization skips a separate bound-only LP within the heuristic. It then solves one full original-model LP relaxation, using barrier without crossover, to obtain the initial node-balance multipliers. Optimal LP dual values are read with the sign convention used in the Lagrangian objective.

The Lagrangian oracle relaxes node flow balances and separates capture, storage and transportation decisions. Capture and storage blocks use Gurobi, while the base transportation oracle solves the independent directed-arc construction problem analytically. A proximal Bundle method updates the multipliers. The neighborhood routines search for feasible improvements through construction delays, timing windows and joint route/facility replacements. When permitted by the stopping condition, remaining Lagrangian time can be used for further primal search after the oracle-evaluation limit is reached. The optional strengthening routines are documented separately in `LRforFNE/STRENGTHENING.md`; they are not enabled by the standard entry points.

An upper bound is the original cost of a complete plan that passes the original-model feasibility audit. A global lower bound is taken from a valid original-model relaxation or the certified oracle bound, with its origin recorded. The objective of a relaxed witness can violate network balances and is not an original-model upper bound. Likewise, neighborhood bounds and Bundle diagnostics are not substituted for global lower bounds. The reported relative gap uses the audited upper bound and certified lower bound.

C. Standalone Primal Heuristic

PH can also solve the original model independently. It first solves the full LP relaxation, using its objective as a global lower bound only when optimality is established. It then constructs an initial feasible plan and improves facility schedules, pipeline construction and routes through the shared `heuristic_stages/` routines.

Only audited feasible plans provide upper bounds; restricted neighborhood bounds do not replace the global lower bound. The default initial-solution backend is RSS, selected by `SP_SOLVER_METHOD`.

Both PH entries default to `LP_TIME_LIMIT=None` and `TIME_LIMIT=3600`. The LP has no finite time limit and is timed separately; the subsequent 3,600-second budget covers initialization, neighborhood search and final polishing. Thus, the configured PH budget is not a limit on the complete run. `MAX_ROUNDS=200` limits search rounds, while stage-specific budgets are also editable at the top of each entry file.

D. Direct Gurobi Solution

The direct interfaces submit the full original mixed-integer model to Gurobi. An available incumbent supplies the upper bound through `ObjVal`, while `ObjBound` supplies the global MIP lower bound. Status and `MIPGap` must be read with these bounds: reaching a time limit does not establish optimality, and a run without an incumbent has no feasible upper bound.

The current single-case `TIME_LIMIT` is 15,000 seconds; the batch limit is 11,000 seconds per case. These limits apply to Gurobi optimization, excluding data loading, model construction and result export. There is no separate PH-style LP budget. The single-case solver sets `MIPFocus=1`, `Heuristics=1`, `Method=2`, `NodeMethod=2` and `Crossover=0`. The batch solver explicitly sets the time limit, output flag and `GUROBI_THREADS=0` for automatic thread selection, leaving other parameters at their solver defaults. The two interfaces therefore have different default solver settings; align them before making controlled runtime comparisons.

IV. Output Files and Result Inspection

A. Lagrangian Single-Case and Batch Records

Lagrangian outputs are stored under `LRforFNE/output/`. The editable single-case entry writes to `run_lrh/bundle/`; normalized runs use `run_lrh/normalized/bundle/`. Each run produces the six structured files in Table S.4 and a `variables.xlsx` workbook. The workbook contains seven variable sheets, a solve summary, model and algorithm parameters, and initialization, iteration and primal-search logs. When no feasible plan is available, only the summary, parameters and existing logs are exported to Excel.

Table S.4—Structured Lagrangian result files

File	Content
`result.json`	Status, termination reason, bounds, gap, timing, instance fingerprint, effective settings, feasibility audit and diagnostics.
`solution.json`	Sparse records for all seven original-model variable groups; each index is stored as an array.
`history.jsonl`	Oracle evaluations, certified intervals, residuals, bound updates and Bundle decisions.
`initialization_history.jsonl`	Feasible-initialization stages and the multiplier-LP record.
`initial_multipliers.json`	LP-derived initial multipliers and their initialization metadata.
`best_multipliers.json`	Multipliers associated with the best certified oracle bound and its evaluation identifier.

The seven variable groups are `s` for capture-unit activation, `r` for storage-site opening, `w` for new wells, `y` for new pipelines, and `a`, `b`, `x` for annual capture, injection and transport flows. The workbook uses facility or arc–class indices followed by `period1`, `period2` and subsequent period columns. Construction entries record additions in that period rather than cumulative installed capacity.

The batch entry writes a `CCS_FNE_LR_bundle_30Y_result.csv` summary beneath `output/batch/`, with full JSON records under `cases/`. Normalized cases use a separate `normalized/` subdirectory. Summary rows contain the six case-identification fields, bounds, gap, status, termination reason, lower-bound origin, runtime components and the detailed-result directory. JSON records are saved before the associated CSV is atomically updated. Batch runs do not generate variable workbooks automatically. The command-line entry `run_lr.py` uses the same batch procedure when `--batch` is specified; its single-case runs write timestamped JSON directories.

B. Primal-Heuristic Results

The PH single-case entry writes `output/CCS_FNE_PH_variables.xlsx`, containing the same seven variable groups, model and algorithm parameters, a solve summary and stage logs. If no feasible plan is found, it saves the summary, parameters and logs without variable sheets or maps. For a feasible plan, the entry invokes the shared plotting routine to generate period-by-period construction maps under `output/CCS_FNE_PH_results_periods/`, subject to the plotting settings.

The PH batch entry writes `output/CCS_FNE_PH_30Y_result.csv`; normalized runs use `output/normalized/`. Each case adds bounds, gap, status, runtime, LP information, lower-bound origin and audit violation to the six identifying fields. The summary is saved after each case; batch PH does not export per-case variable workbooks or maps. Existing historical result files are not renamed.

C. Direct Gurobi Results

The Gurobi single-case entry writes `output/CCS_FNE_gurobi_variables.xlsx`. A feasible run includes the seven variable sheets, `model_parameters`, `algorithm_parameters`, `solve_summary` and `solve_log`. The summary records status, incumbent objective, lower bound, gap and Gurobi runtime. The log sheet retains solver messages even when `GUROBI_OUTPUT_FLAG=0` suppresses console output. Without an incumbent, the workbook contains parameters, summary and logs but no variable sheets or maps. Feasible runs invoke the shared plotting routine for construction maps under `output/CCS_FNE_gurobi_results_periods/`, subject to its settings.

The Gurobi batch entry atomically updates `output/CCS_FNE_30Y_result.csv` after each case; normalized runs use `output/normalized/`. The CSV contains the six identifying fields, final `UB`, `LB` and `gap`, two bound snapshots and `status`. It does not store runtime, per-case variable workbooks or solver logs. The historical snapshot column prefixes `2h` and `5h` are retained, but the current `RECORD_TIMES` thresholds are 3,600 and 10,800 seconds. Snapshots are captured at the first MIP callback at or after each threshold; if no snapshot was captured before termination, the final metrics fill that slot. Interpret these columns using the configured thresholds rather than their names.

D. Visualization and Timing Interpretation

The shared `plot_results.py` module reads construction decisions to draw period-by-period network development. Its `plot_results_from_excel()` function accepts the `variable_s`, `variable_r`, `variable_w`

and variable_y sheets written by the single-case entry. Pipeline capacity is computed from the installed class capacities and integer pipeline counts. The module also provides plot_bound_history() for bound trajectories. Visualization is a separate step; the Lagrangian entry points save their results without automatically generating maps.

Initialization, multiplier-LP and Lagrangian runtimes should be read together. The multiplier LP is outside the Bundle Lagrangian deadline, and the reported total is therefore not limited to the configured Lagrangian time alone. The structured logs record stages and iterations, rather than the complete console text of every Gurobi subproblem. Final primal search can improve the upper bound after the last oracle record, so the final result summary is the authoritative end-of-run value.

V. Reproduction and Verification

A. Running the Lagrangian Interfaces

For a single case, edit the model and algorithm blocks in run_lrh.py, including the period count, NP, pipeline catalogue, target profile and budgets, then execute the first command below. For the shared grid, edit the algorithm settings in CCS_FNE_LR.py and the model grid in experiment_setup.py, then execute the second command. The command-line interface is available for explicit parameter overrides.

```
python -B LRforFNE/run_lrh.py
python -B LRforFNE/CCS_FNE_LR.py
python -B -m LRforFNE.run_lr --help
```

The batch runner skips cases recorded as normally completed in the Bundle summary. Its completion set includes certified-gap, feasible time-limit, feasible iteration-limit, feasible stagnation and proven-infeasibility statuses. Completion of a budgeted run does not imply global optimality. Failed initialization, solver errors and interruption remain eligible for retry. Cases are matched by period count, catalogue size, NP, maximum capture target, curve and coefficient. A new result directory should be used when changing data, same-size catalogue contents or algorithm settings, because those changes are not detected by this resume key. Retried cases initialize afresh rather than restoring internal iterations.

B. Running the Primal-Heuristic Interfaces

For one PH case, edit the model and algorithm settings in run_PH.py, then run the first command below. Its defaults are 15 periods, NP equal to three, 12 pipeline classes, a linear profile and a terminal annual target of 1,000,000 kt CO_2/year. The main model controls are PERIOD_NUM, NP, DIAMETERS, CURVE, COEFFICIENT and MAX_CAPTURE_AMOUNT. An explicit CAPTURE_AMOUNT_SEQUENCE overrides the generated target; NORMALIZE_CAPTURE_TARGET controls nonlinear-profile normalization.

For batch PH, edit the algorithm settings in CCS_FNE_PH.py and the shared model grid in experiment_setup.py, then run the second command. It executes the same 224-case grid as the other batch solvers, independently of the single-case settings. These are configuration-file interfaces; parameters are edited in the scripts.

```
python -B run_PH.py
python -B CCS_FNE_PH.py
```

Batch PH resumes from its summary using the six case-identification fields and the entry's COMPLETED_STATUSES; failed or interrupted cases are retried. It retains read-only compatibility with the historical CCS_FNE_H_30Y_checkpoint.csv. When changing data or algorithm settings, use a new RESULT_DIR to isolate both the result and checkpoint paths. For Python callers, both PH modules expose build_options() for configuration and solve_heuristic() for solving a prepared instance.

C. Running the Direct Gurobi Interfaces

For one Gurobi case, edit the model and algorithm settings in run_gurobi.py, then execute the first command below. The default model matches the PH single-case settings: 15 periods, NP equal to three, 12 pipeline classes, a linear profile and a terminal annual target of 1,000,000 kt CO_2/year. The model controls have the same names as in run_PH.py, including CAPTURE_AMOUNT_SEQUENCE and NORMALIZE_CAPTURE_TARGET. Set TIME_LIMIT and GUROBI_OUTPUT_FLAG in the algorithm block; the additional Gurobi parameters are assigned inside solve_original_model().

For batch Gurobi, edit TIME_LIMIT, GUROBI_THREADS, GUROBI_OUTPUT_FLAG and the snapshot thresholds in CCS_FNE.py, and edit the shared grid in experiment_setup.py. The second command runs the same 224-case grid in sequence, independently of the single-case settings. Both are configuration-file interfaces, with parameters edited in the scripts.

```
python -B run_gurobi.py
python -B CCS_FNE.py
```

Batch Gurobi matches the six identifying fields against its result CSV and skips statuses normalized to optimal, time_limit or root_optimal; other statuses are retried. A time-limit row is treated as completed even without an incumbent, so inspect its upper bound before treating it as a feasible result. Use a new RESULT_DIR when changing data, catalogue contents or solver settings. For Python callers, run_gurobi.solve_original_model() returns variables and detailed run records, while CCS_FNE.solve_milp() returns final and snapshot metrics; its return_solution=True option additionally returns the seven variable groups.

D. Verification and Data Preparation Checks

The small mathematical fixture exercises initialization and the Bundle method without loading the national network. The following PowerShell commands run that check and the test suite:

```
python -B -m LRforFNE.run_lr --tiny --method bundle `
  --max-iterations 12 --lr-time-limit 20 `
  --warm-start-time-limit 10 --lp-time-limit 5 --quiet
New-Item -ItemType Directory -Force tmp | Out-Null
python -B -m pytest -q -p no:cacheprovider `
  --basetemp=tmp/lr_tests LRforFNE/tests
```

For a numerical comparison, match the same processed inputs, periods, pipeline catalogue, NP and capture-target sequence in the PH, Lagrangian and direct Gurobi entries. Check the saved parameters, the Lagrangian instance fingerprint, the feasibility audit, the origin of the lower bound and the actual termination status. Capture targets and capacities must use the same annual units, and cumulative quantities must include the period durations. Bounds and runtime claims should be read from the corresponding run records rather than inferred from construction-variable tables alone.

To regenerate the derived inputs, follow the local preparation guides `source_cluster/generate_source_hub.md`, `storage_cost/sink_subsite/generate_sink_subsite.md`, `storage_cost/storage_cost.md`, `triangle network/network_generate.md` and `triangle network/pipe_cost_calculation.md`. Source clustering and storage-subsite construction precede network generation; pipeline costs depend on the resulting arcs. Storage costs are computed from the subsite and engineering inputs. Regeneration begins from the supplied source-unit table, basin data, boundaries, engineering workbooks and reference tables. The fixed `pipeline_capacity.csv` catalogue is supplied directly and is not generated by these scripts. After synchronization to the project root, the model loader provides the final consistency checks before solving.

The project repository is available at `https://github.com/tm150912/CCS_FNE_LRH`. Instructions for the Lagrangian entry points, output formats and optional strengthening procedures are provided in `LRforFNE/README.md` and `LRforFNE/STRENGTHENING.md`.